Air University

Department of Computer Science

CS426L Introduction to Parallel & Distributed Computing

Lab 08: Distributed-Memory Computing, Message Passing & Distributed-System Architectures

Lab Report (C / Open MPI)

Name:	**Reg. No:** _____
_____	
Section: VII C (FALL26)	**Instructor:** Muhammad Nauman Zafar Hashmi

Setup

I used WSL (Ubuntu) on Windows 11 with GCC 15.2.0 and Open MPI 5.0.10 (Ubuntu `openmpi-bin` / `libopenmpi-dev` packages). All files are in `~/PDC_Lab08`. Logical cores (`nproc`): **8**.

```
$ mpicc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ ompi_info | grep "Open MPI:"
                Open MPI: 5.0.10
$ mpirun -np 2 hostname
DESKTOP-IIC2T8E
DESKTOP-IIC2T8E
$ nproc
8
```

Task 1: Verify MPI and Run Hello World

Each rank asks for its rank, the communicator size and the processor name and prints a line straight away (the "unordered" part). After a barrier every rank except 0 sends its line to rank 0 with `MPI_Send`, and rank 0 receives them one by one from rank 1, 2, 3 ... and prints them (the "ordered" report).

Source code: hello_mpi.c

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    int rank, size, len;
    char name[MPI_MAX_PROCESSOR_NAME];
    char line[MPI_MAX_PROCESSOR_NAME + 64];

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(name, &len);

    // unordered: every rank prints whenever it gets here
```

```
    printf("Hello from rank %d of %d on %s\n", rank, size, name);
    fflush(stdout);

    MPI_Barrier(MPI_COMM_WORLD);

    // ordered: every rank sends its line to rank 0, rank 0 prints in order
    snprintf(line, sizeof(line), "rank %d of %d on %s", rank, size, name);
    if (rank == 0) {
        printf("--- ordered report from rank 0 ---\n");
        printf("%s\n", line);
        for (int r = 1; r < size; r++) {
            MPI_Recv(line, sizeof(line), MPI_CHAR, r, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("%s\n", line);
        }
    } else {
        MPI_Send(line, strlen(line) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Results

Ranks started	Communicator size reported	Order of ranks, run 1 (unordered)	Order of ranks, run 2 (unordered)	Processor name(s)
4	4	2 1 0 3	0 1 3 2	DESKTOP-IIC2T8E
6	6	4 0 2 3 1 5	1 0 2 5 3 4	DESKTOP-IIC2T8E
8	8	7 6 5 0 2 4 1 3	4 1 5 6 3 0 7 2	DESKTOP-IIC2T8E

MPI implementation and version: **Open MPI 5.0.10** Logical cores (nproc): **8**

Unordered vs ordered: the "Hello from rank ..." lines came out in a different order every time, and sometimes a hello line even showed up after the ordered header (run 1 with 4 ranks). The ordered report was always 0, 1, 2, ... n-1 in all six runs.

Discussion questions

1. **MPI_Init / MPI_Finalize.** `MPI_Init` starts the MPI library in the process: it connects the process to the other processes started by mpirun, sets up `MPI_COMM_WORLD` and gives the process its rank. `MPI_Finalize` cleans everything up and closes the connections, and no MPI call may be made after it. If you call MPI functions before Init or after Finalize the behaviour is undefined, and in practice Open MPI prints an error and aborts the program.
2. **Rank, size, MPI_COMM_WORLD.** `MPI_COMM_WORLD` is the default communicator that contains all processes started together. The size is how many processes are in it and the rank is the ID of one process inside it (0 to size-1). A process uses them to pick its share of the work, e.g. in my sum every rank takes the chunk from `rank*N/size` to `(rank+1)*N/size`, and rank 0 is the one that collects and prints.

3. **Why the order changed.** The ranks are separate processes, and the OS schedules them independently, so whichever one reaches `printf` first prints first. Their output also goes through mpirun, which forwards it whenever it arrives, so the order is basically random. The ordered report is always in order because only rank 0 prints it, and it calls `MPI_Recv` for source 1, then 2, then 3 and so on. A blocking receive from a specific source forces the order, so the messages are what made the difference.
4. **Same processor name.** All the ranks run on my one laptop, so they all report the same hostname (DESKTOP-IIC2T8E). On a cluster with several nodes, ranks would be spread over the nodes and I would expect different names, like node01 for ranks 0-3 and node02 for ranks 4-7, depending on how many slots each node has.
5. **./hello_mpi vs mpirun -np 4.** Running `./hello_mpi` directly starts a single process (a "singleton"), so size is 1 and only rank 0 prints. `mpirun -np 4` launches 4 copies of the program, connects them together and gives them ranks 0-3, so you get 4 processes that can talk to each other.

Task 2: Message Passing vs Shared Memory

Both programs add up an array of 10,000,000 doubles (all 1.0, so the answer must be 10,000,000). In the MPI program only rank 0 has the array. The **collective** variant uses `MPI_Scatter` + `MPI_Reduce`, and the **point-to-point** variant has rank 0 `MPI_Send` each chunk by hand and then `MPI_Recv` each partial sum. The OpenMP version just uses `#pragma omp parallel for reduction(+:sum)`. Each timing is the average of 3 repetitions.

Source code: sum_mpi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 10000000
#define REPS 3

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (N % size != 0) {
        if (rank == 0) printf("number of ranks must divide %d\n", N);
        MPI_Finalize();
        return 1;
    }

    int chunk = N / size;
    double *data = NULL;
    double *part = malloc(chunk * sizeof(double));

    // only rank 0 has the full array
    if (rank == 0) {
        data = malloc(N * sizeof(double));
        for (int i = 0; i < N; i++) data[i] = 1.0;
    }

    double expected = N;
    double t_coll = 0, t_p2p = 0;
    double total_coll = 0, total_p2p = 0;

    for (int rep = 0; rep < REPS; rep++) {
        // collective variant: Scatter + Reduce
        MPI_Barrier(MPI_COMM_WORLD);
        double t0 = MPI_Wtime();

        MPI_Scatter(data, chunk, MPI_DOUBLE, part, chunk, MPI_DOUBLE, 0, MPI_COMM_WORLD);
        double local = 0;
        for (int i = 0; i < chunk; i++) local += part[i];
        MPI_Reduce(&local, &total_coll, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

        t_coll += MPI_Wtime() - t0;

        // point-to-point variant: Send / Recv by hand
```

```
        MPI_Barrier(MPI_COMM_WORLD);
        t0 = MPI_Wtime();

        if (rank == 0) {
            for (int r = 1; r < size; r++)
                MPI_Send(data + r * chunk, chunk, MPI_DOUBLE, r, 0, MPI_COMM_WORLD);
            local = 0;
            for (int i = 0; i < chunk; i++) local += data[i];
            total_p2p = local;
            for (int r = 1; r < size; r++) {
                double other;
                MPI_Recv(&other, 1, MPI_DOUBLE, r, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                total_p2p += other;
            }
        } else {
            MPI_Recv(part, chunk, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            local = 0;
            for (int i = 0; i < chunk; i++) local += part[i];
            MPI_Send(&local, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD);
        }

        t_p2p += MPI_Wtime() - t0;
    }

    if (rank == 0) {
        printf("ranks=%d\n", size);
        printf("collective     sum=%.0f  avg=%.2f ms  correct=%s\n",
               total_coll, t_coll / REPS * 1000, total_coll == expected ? "YES" : "NO");
        printf("point-to-point sum=%.0f  avg=%.2f ms  correct=%s\n",
               total_p2p, t_p2p / REPS * 1000, total_p2p == expected ? "YES" : "NO");
        free(data);
    }

    free(part);
    MPI_Finalize();
    return 0;
}
```

Source code: sum_omp.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 10000000
#define REPS 3

int main(void)
{
    double *data = malloc(N * sizeof(double));
    for (int i = 0; i < N; i++) data[i] = 1.0;

    double sum = 0, t = 0;

    for (int rep = 0; rep < REPS; rep++) {
        double t0 = omp_get_wtime();
        sum = 0;

        // all threads read the same array, no copying needed
        #pragma omp parallel for reduction(+:sum)
```

```
        for (int i = 0; i < N; i++) sum += data[i];

        t += omp_get_wtime() - t0;
    }

    printf("threads=%d\n", omp_get_max_threads());
    printf("reduction sum=%.0f  avg=%.2f ms  correct=%s\n",
           sum, t / REPS * 1000, sum == N ? "YES" : "NO");

    free(data);
    return 0;
}
```

Timings (average of 3 repetitions, ms)

Ranks / threads	MPI collective	MPI point-to-point	OpenMP reduction	All correct? (Y/N)
1	67.19	20.95	16.12	Y
2	45.54	15.11	7.55	Y
4	33.61	18.08	4.51	Y
8	56.64	25.31	8.67	Y

Comparison of the two models

Aspect	Shared memory (OpenMP / TPL)	Message passing (MPI)
Memory: who can see which data	All threads see the same address space, so every thread can read the whole array.	Each process has its own private memory. Rank 1 cannot see rank 0's array at all.
How a worker gets its data	It just indexes the shared array; OpenMP splits the loop iterations.	The data has to be sent to it (Scatter, or Send/Recv), which copies it into its own buffer.
How partial results are combined	`reduction(+:sum)` gives each thread a private sum and adds them at the end of the loop.	`MPI_Reduce`, or every rank sends its partial sum to rank 0 and rank 0 adds them.
Synchronization	Implicit barrier at the end of the parallel loop; locks/atomics if threads write shared data.	Through the messages themselves (a blocking Recv waits for its data), plus barriers and collectives.
Scaling beyond one machine	Not possible, limited to the cores of one machine.	Yes, the same code runs on many nodes connected by a network.
Effect of one failed worker	A crashed thread brings down the whole process.	By default the whole MPI job is aborted too, but the processes are separate, so in principle the failure can be isolated.
Main cost seen in this lab	Thread start-up and memory bandwidth (8 threads were slower than 4).	Copying/sending the data to the ranks; this was bigger than the summing itself.

Written comparison

In the shared-memory model all the threads live inside one process, so the OpenMP version only needed one pragma and every thread read its part of the same array directly. MPI follows the message-passing paradigm instead: each rank is a separate process with its own private memory, and the only way to share data is to send and receive explicit messages. That is why rank 0 first had to scatter the array before anyone could start

adding, and why the partial sums had to travel back as messages too. On my machine OpenMP was clearly the fastest (4.51 ms with 4 threads), while both MPI variants spent most of their time moving the data around rather than adding numbers. The collective variant was even slower than point-to-point here, mostly because `MPI_Scatter` also copies rank 0's own chunk into a new buffer, while in my point-to-point code rank 0 just sums its part in place. Adding more ranks barely helped and 8 ranks was slower again, because more ranks means more messages and all of them compete for the same memory bandwidth. Still, message passing has the big advantage that it does not need shared memory, so the same program could run across many machines where OpenMP cannot. So on one PC shared memory wins, but message passing is what you need once the work no longer fits on one machine.

Discussion questions

1. **Why scatter?** The OpenMP threads share one address space, so the array is already "there" for all of them. MPI ranks are different processes with separate memory; only rank 0 created the array, so the other ranks literally do not have the data until rank 0 sends it. This shows that MPI uses a distributed-memory model: no shared variables, and all sharing is by explicit messages.
2. **Fastest variant.** OpenMP reduction was fastest at every count (best 4.51 ms at 4 threads). The MPI variants do extra work: copying 80 MB of data into messages/buffers, sending it to each rank, the receive side copying it out again, and sending the partial sums back. Adding ranks did not really reduce the time (collective went 67 → 46 → 34 ms and then up to 57 ms at 8; point-to-point stayed around 15-25 ms), because the communication cost is about as big as the work, and the summing itself is so cheap that splitting it saves very little.
3. **Send/receive semantics.** A message is delivered when the receive matches the send on the communicator (same one), the source (the receive names the sender or uses `MPI_ANY_SOURCE`), the destination (the receiver's rank is the one in the send) and the tag (same tag or `MPI_ANY_TAG`). The type and count must also fit the buffer. Blocking means the call does not return until it is safe: `MPI_Recv` waits until the data has actually arrived in the buffer, and `MPI_Send` waits until the send buffer can be reused (the data is copied out or received), so a send without a matching receive can hang the program.
4. **Collective vs point-to-point.** Both gave the right answer. The point-to-point code is longer, rank 0 has to loop over all ranks itself, and it is easy to get tags or counts wrong and deadlock. Collectives are one line, everyone calls the same function, and the MPI library can use better algorithms internally (like tree-based reduce, which takes log(p) steps instead of p-1 messages to rank 0). On a real cluster that matters a lot, even though on my laptop the collective was slower because of the extra copy.
5. **1,000 nodes.** Each node of a cluster has its own RAM, and there is no memory that all 1,000 nodes can access directly, so OpenMP threads simply cannot run across them. Message passing gives you processes on every node that exchange data over the network, plus collectives that scale to thousands of ranks. So you can use all the nodes' CPUs and memory together.

Task 3: MPI Deployments and Architecture Styles

master_worker.c is a task farm: rank 0 is the master (server) with 40 tasks (task t = compute t^2). It gives each worker one task, and whenever a result comes back it sends that worker the next task, until none are left and it sends STOP. **ring.c** is a token ring: every rank only talks to its neighbours, and the token goes around 1000 times. Each rank adds 1 before passing it on, so after a lap rank 0 should get back size-1.

Source code: master_worker.c

```
#include <stdio.h>
#include <mpi.h>

#define TASKS 40
#define TAG_WORK 1
#define TAG_STOP 2

// the "work": square of the task number
int do_task(int t)
{
    return t * t;
}

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        // master = server, hands out tasks one by one
        int served[64] = {0};
        long total = 0;
        int next = 0, active = 0, result;
        MPI_Status st;

        // give every worker its first task
        for (int w = 1; w < size; w++) {
            if (next < TASKS) {
                MPI_Send(&next, 1, MPI_INT, w, TAG_WORK, MPI_COMM_WORLD);
                next++;
                active++;
            } else {
                MPI_Send(&next, 1, MPI_INT, w, TAG_STOP, MPI_COMM_WORLD);
            }
        }

        // whoever finishes gets the next task
        while (active > 0) {
            MPI_Recv(&result, 1, MPI_INT, MPI_ANY_SOURCE, TAG_WORK, MPI_COMM_WORLD, &st);
            int w = st.MPI_SOURCE;
            total += result;
            served[w]++;
            active--;

            if (next < TASKS) {
                MPI_Send(&next, 1, MPI_INT, w, TAG_WORK, MPI_COMM_WORLD);
```

```
                next++;
                active++;
            } else {
                MPI_Send(&next, 1, MPI_INT, w, TAG_STOP, MPI_COMM_WORLD);
            }
        }

        long expected = 0;
        for (int t = 0; t < TASKS; t++) expected += (long)t * t;

        printf("ranks=%d workers=%d tasks=%d\n", size, size - 1, TASKS);
        for (int w = 1; w < size; w++)
            printf("worker %d served %d tasks\n", w, served[w]);
        printf("total=%ld expected=%ld correct=%s\n", total, expected,
               total == expected ? "YES" : "NO");
    } else {
        // worker = client, asks for work until told to stop
        int task;
        MPI_Status st;
        while (1) {
            MPI_Recv(&task, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &st);
            if (st.MPI_TAG == TAG_STOP) break;
            int result = do_task(task);
            MPI_Send(&result, 1, MPI_INT, 0, TAG_WORK, MPI_COMM_WORLD);
        }
    }

    MPI_Finalize();
    return 0;
}
```

Source code: ring.c

```
#include <stdio.h>
#include <mpi.h>

#define LAPS 1000

int main(int argc, char **argv)
{
    int rank, size, token;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // every rank only talks to its two neighbours
    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    for (int lap = 0; lap < LAPS; lap++) {
        if (rank == 0) {
            token = 0;
            MPI_Send(&token, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
            MPI_Recv(&token, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        } else {
            MPI_Recv(&token, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            token++;   // each rank adds 1 before passing it on
```

```
            MPI_Send(&token, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
        }
    }

    double t = (MPI_Wtime() - t0) * 1000;

    if (rank == 0) {
        long messages = (long)LAPS * size;
        printf("ranks=%d laps=%d messages=%ld\n", size, LAPS, messages);
        printf("token after last lap=%d expected=%d\n", token, size - 1);
        printf("time=%.3f ms  per message=%.5f ms  correct=%s\n",
               t, t / messages, token == size - 1 ? "YES" : "NO");
    }

    MPI_Finalize();
    return 0;
}
```

Task farm (client-server style)

Ranks	Workers	Tasks served per worker	Correct? (Y/N)
4	3	W1 = 14, W2 = 13, W3 = 13 (total 40)	Y (sum = 20540)
8	7	W1 = 7, W2 = 7, W3 = 1, W4 = 7, W5 = 6, W6 = 6, W7 = 6 (total 40)	Y (sum = 20540)

The split depends on which worker answers first, so it changes from run to run. In the 8-rank run worker 3 got only one task, probably because its process was scheduled late and the others had already taken the rest. That is the dynamic load balancing working.

Token ring (peer-to-peer style)

Ranks	Time (ms)	Time per message (ms)	Correct? (Y/N)
4	1.024	0.00026 (4000 messages)	Y
8	1098.059	0.13726 (8000 messages)	Y

The 8-rank time jumped a lot. In an earlier test run the same 8-rank ring took only 5.166 ms (0.00065 ms per message). With 8 ranks on 8 logical cores, every core is busy, and Open MPI ranks busy-wait (spin) while waiting for a message. So whenever Windows/WSL needs a core, one rank of the ring gets descheduled and everyone behind it has to wait. A ring is only as fast as its slowest link.

Architecture styles compared for an MPI cluster

Style	How my MPI programs / cluster map onto it	Strengths	Weaknesses	Fit (1-5)
Client-server	master_worker: rank 0 is the server holding the task list, the workers act as clients asking for work. Also the login node / job scheduler is a server that users submit jobs to.	Simple to write, automatic load balancing, easy to collect results in one place.	Rank 0 is a bottleneck and a single point of failure; it does not scale to thousands of workers.	3
Peer-to-peer	ring.c, and most real MPI codes (SPMD): every rank runs the same program and talks directly to neighbours; collectives like Allreduce involve all ranks as equals.	No central bottleneck, scales well, communication stays between neighbours,	Harder to balance load and to program; one slow rank holds	4

		uses the fast interconnect directly.	everyone up (seen in my 8-rank ring).	
Layered / tiered	User/login tier → scheduler (Slurm) tier → compute nodes running mpirun/MPI library → network/transport layer (InfiniBand/Ethernet). Each layer only uses the one below it.	Clear separation: the program does not care about the network hardware; each layer can be changed on its own.	Describes the whole system, not how ranks talk; every layer adds a bit of overhead.	5

Design note: deploying MPI programs on a cluster

1. The deployment. I am imagining a small cluster of 16 compute nodes, each with 2 sockets × 16 cores (32 cores), so 32 ranks per node and 512 ranks in total. The nodes are connected by an InfiniBand switch for MPI traffic and a separate Ethernet network for management and the shared file system (NFS). Users log in over SSH to one login node, which has the compilers and Open MPI. They never run jobs on it; they submit a job script to the Slurm scheduler, which picks free nodes and starts `mpirun`/`srun` with one rank per core.

2. Client-server mapping. At the system level the login node and Slurm controller are servers: users (clients) send them job requests and they answer with an allocation. Inside a program, my `master_worker` is client-server: rank 0 serves tasks and collects results, and the workers request work. This is nice for many independent tasks of different sizes, but every message goes through rank 0, so with hundreds of workers rank 0 becomes the bottleneck. And if it dies the whole job is lost.

3. Peer-to-peer mapping. A typical MPI program is SPMD: all 512 ranks run the same code, own a part of the data, and exchange data directly with neighbours (like the ring, or halo exchange in a grid). Collectives like `MPI_Allreduce` are done by all ranks together without a central owner. There is no single hotspot, so it scales, but every rank depends on its neighbours, and the speed is set by the slowest rank and by network latency.

4. Layered / tiered mapping. From top to bottom: (a) the user tier — login node, editor, job scripts; (b) the resource-management tier — Slurm decides where and when the job runs; (c) the application tier — the MPI program on the compute nodes; (d) the middleware layer — the MPI library and runtime (PRRTE/PMIx) that start the processes, assign ranks and provide Send/Recv/collectives; (e) the transport/network layer — shared memory inside a node, InfiniBand/TCP between nodes. Each layer hides the one below it, which is why my code ran unchanged on a laptop and would also run on the cluster.

5. Comparison. Client-server is the simplest but centralised; peer-to-peer is the most scalable for the actual number crunching; layered describes how the whole cluster is built and managed. They are not exclusive: a real cluster is layered overall, the management parts are client-server, and the compute part is mostly peer-to-peer.

6. Recommendation. I would describe an MPI cluster as a **layered/tiered system** whose compute layer uses a **peer-to-peer (SPMD)** style. Reasons: (1) the layering is what makes the cluster usable and portable, since users, the scheduler, the MPI runtime and the network each have a clear job; (2) peer-to-peer communication between ranks avoids the rank-0 bottleneck and uses the interconnect best, which is why collectives and neighbour exchange scale to thousands of ranks. Client-server is still the better choice for part of the system: the job scheduler/login node, and programs made of many independent tasks of uneven size (parameter sweeps), where a master-worker farm balances the load automatically.

Discussion questions

1. **Client-server program.** `master_worker.c`. Rank 0 is the server/master: it owns the task list, hands out tasks, receives the results and adds them up. Ranks 1..n-1 are clients/workers: they wait for a task, compute it and send back the result. The bottleneck is rank 0: every task needs two messages through it, so with many workers or very short tasks workers end up waiting in a queue at rank 0, and its network link and CPU become the limit.
2. **Peer-to-peer program.** `ring.c`. Every rank runs the same code and has the same role: receive from the left neighbour, add one, send to the right neighbour (rank 0 only starts each lap). No rank is in charge of the others. The speed is limited by latency, because the messages go strictly one after another, so time = laps × ranks × latency. It is also limited by the slowest rank: in my 8-rank run one descheduled process slowed the whole ring down.
3. **Layered view.** User layer (login node: edit, compile, submit job) → scheduler layer (Slurm: queues jobs, allocates nodes) → application layer (the MPI program, one rank per core) → MPI library/runtime layer (launches ranks, matches messages, implements collectives) → OS and transport layer (shared memory, TCP or InfiniBand drivers) → physical network (cables and switches). Each layer only talks to the layer right below it.
4. **Best style.** Layered overall with peer-to-peer compute (see recommendation). Reasons: the layers separate concerns and make programs portable between a laptop and a cluster; peer-to-peer has no central bottleneck so it scales; it also matches the SPMD model that MPI is built around. A different style is better for job submission (client-server with the scheduler), and for embarrassingly parallel task farms where master-worker gives simple dynamic load balancing.
5. **Node failure.** Client-server: if a worker dies, the master could in principle resend its task to another worker, but if the master dies everything is lost. Peer-to-peer: a dead rank breaks the ring/neighbour exchange, so the others block forever waiting for it. Layered: a failed compute node is handled by the scheduler layer (mark the node down, requeue the job). By default MPI uses `MPI_ERRORS_ARE_FATAL`: if one rank fails, the runtime kills the whole job. So inside one MPI job the style does not save you from a crash. Fault tolerance has to come from the layer above (checkpoint/restart and job requeue), and a master-worker design only helps if you change the error handler and write your own recovery.

Appendix: Build and Run Output

Complete terminal output of building and running all programs (`run_all.sh`). All runs used `mpirun --oversubscribe`.

```
### verify
$ mpicc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ ompi_info | grep "Open MPI:"
                Open MPI: 5.0.10
$ mpirun -np 2 hostname
DESKTOP-IIC2T8E
DESKTOP-IIC2T8E
$ nproc
8
### build
$ mpicc -O2 -std=gnu11 hello_mpi.c -o hello_mpi
  ok
$ mpicc -O2 -std=gnu11 sum_mpi.c -o sum_mpi
  ok
$ gcc -O2 -fopenmp -std=gnu11 sum_omp.c -o sum_omp
  ok
$ mpicc -O2 -std=gnu11 master_worker.c -o master_worker
  ok
$ mpicc -O2 -std=gnu11 ring.c -o ring
  ok
### task1
$ mpirun -np 4 ./hello_mpi   (run 1)
Hello from rank 2 of 4 on DESKTOP-IIC2T8E
Hello from rank 1 of 4 on DESKTOP-IIC2T8E
Hello from rank 0 of 4 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 4 on DESKTOP-IIC2T8E
rank 1 of 4 on DESKTOP-IIC2T8E
rank 2 of 4 on DESKTOP-IIC2T8E
rank 3 of 4 on DESKTOP-IIC2T8E
Hello from rank 3 of 4 on DESKTOP-IIC2T8E
$ mpirun -np 4 ./hello_mpi   (run 2)
Hello from rank 0 of 4 on DESKTOP-IIC2T8E
Hello from rank 1 of 4 on DESKTOP-IIC2T8E
Hello from rank 3 of 4 on DESKTOP-IIC2T8E
Hello from rank 2 of 4 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 4 on DESKTOP-IIC2T8E
rank 1 of 4 on DESKTOP-IIC2T8E
rank 2 of 4 on DESKTOP-IIC2T8E
rank 3 of 4 on DESKTOP-IIC2T8E
$ mpirun -np 6 ./hello_mpi   (run 1)
Hello from rank 4 of 6 on DESKTOP-IIC2T8E
Hello from rank 0 of 6 on DESKTOP-IIC2T8E
Hello from rank 2 of 6 on DESKTOP-IIC2T8E
Hello from rank 3 of 6 on DESKTOP-IIC2T8E
Hello from rank 1 of 6 on DESKTOP-IIC2T8E
Hello from rank 5 of 6 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 6 on DESKTOP-IIC2T8E
rank 1 of 6 on DESKTOP-IIC2T8E
rank 2 of 6 on DESKTOP-IIC2T8E
```

```
rank 3 of 6 on DESKTOP-IIC2T8E
rank 4 of 6 on DESKTOP-IIC2T8E
rank 5 of 6 on DESKTOP-IIC2T8E
$ mpirun -np 6 ./hello_mpi   (run 2)
Hello from rank 1 of 6 on DESKTOP-IIC2T8E
Hello from rank 0 of 6 on DESKTOP-IIC2T8E
Hello from rank 2 of 6 on DESKTOP-IIC2T8E
Hello from rank 5 of 6 on DESKTOP-IIC2T8E
Hello from rank 3 of 6 on DESKTOP-IIC2T8E
Hello from rank 4 of 6 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 6 on DESKTOP-IIC2T8E
rank 1 of 6 on DESKTOP-IIC2T8E
rank 2 of 6 on DESKTOP-IIC2T8E
rank 3 of 6 on DESKTOP-IIC2T8E
rank 4 of 6 on DESKTOP-IIC2T8E
rank 5 of 6 on DESKTOP-IIC2T8E
$ mpirun -np 8 ./hello_mpi   (run 1)
Hello from rank 7 of 8 on DESKTOP-IIC2T8E
Hello from rank 6 of 8 on DESKTOP-IIC2T8E
Hello from rank 5 of 8 on DESKTOP-IIC2T8E
Hello from rank 0 of 8 on DESKTOP-IIC2T8E
Hello from rank 2 of 8 on DESKTOP-IIC2T8E
Hello from rank 4 of 8 on DESKTOP-IIC2T8E
Hello from rank 1 of 8 on DESKTOP-IIC2T8E
Hello from rank 3 of 8 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 8 on DESKTOP-IIC2T8E
rank 1 of 8 on DESKTOP-IIC2T8E
rank 2 of 8 on DESKTOP-IIC2T8E
rank 3 of 8 on DESKTOP-IIC2T8E
rank 4 of 8 on DESKTOP-IIC2T8E
rank 5 of 8 on DESKTOP-IIC2T8E
rank 6 of 8 on DESKTOP-IIC2T8E
rank 7 of 8 on DESKTOP-IIC2T8E
$ mpirun -np 8 ./hello_mpi   (run 2)
Hello from rank 4 of 8 on DESKTOP-IIC2T8E
Hello from rank 1 of 8 on DESKTOP-IIC2T8E
Hello from rank 5 of 8 on DESKTOP-IIC2T8E
Hello from rank 6 of 8 on DESKTOP-IIC2T8E
Hello from rank 3 of 8 on DESKTOP-IIC2T8E
Hello from rank 0 of 8 on DESKTOP-IIC2T8E
Hello from rank 7 of 8 on DESKTOP-IIC2T8E
Hello from rank 2 of 8 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 8 on DESKTOP-IIC2T8E
rank 1 of 8 on DESKTOP-IIC2T8E
rank 2 of 8 on DESKTOP-IIC2T8E
rank 3 of 8 on DESKTOP-IIC2T8E
rank 4 of 8 on DESKTOP-IIC2T8E
rank 5 of 8 on DESKTOP-IIC2T8E
rank 6 of 8 on DESKTOP-IIC2T8E
rank 7 of 8 on DESKTOP-IIC2T8E
### task2
$ mpirun -np 1 ./sum_mpi
ranks=1
collective     sum=10000000  avg=67.19 ms  correct=YES
point-to-point sum=10000000  avg=20.95 ms  correct=YES
$ mpirun -np 2 ./sum_mpi
ranks=2
collective     sum=10000000  avg=45.54 ms  correct=YES
point-to-point sum=10000000  avg=15.11 ms  correct=YES
```

```
$ mpirun -np 4 ./sum_mpi
ranks=4
collective     sum=10000000  avg=33.61 ms  correct=YES
point-to-point sum=10000000  avg=18.08 ms  correct=YES
$ mpirun -np 8 ./sum_mpi
ranks=8
collective     sum=10000000  avg=56.64 ms  correct=YES
point-to-point sum=10000000  avg=25.31 ms  correct=YES
$ OMP_NUM_THREADS=1 ./sum_omp
threads=1
reduction sum=10000000  avg=16.12 ms  correct=YES
$ OMP_NUM_THREADS=2 ./sum_omp
threads=2
reduction sum=10000000  avg=7.55 ms  correct=YES
$ OMP_NUM_THREADS=4 ./sum_omp
threads=4
reduction sum=10000000  avg=4.51 ms  correct=YES
$ OMP_NUM_THREADS=8 ./sum_omp
threads=8
reduction sum=10000000  avg=8.67 ms  correct=YES
### task3
$ mpirun -np 4 ./master_worker
ranks=4 workers=3 tasks=40
worker 1 served 14 tasks
worker 2 served 13 tasks
worker 3 served 13 tasks
total=20540 expected=20540 correct=YES
$ mpirun -np 8 ./master_worker
ranks=8 workers=7 tasks=40
worker 1 served 7 tasks
worker 2 served 7 tasks
worker 3 served 1 tasks
worker 4 served 7 tasks
worker 5 served 6 tasks
worker 6 served 6 tasks
worker 7 served 6 tasks
total=20540 expected=20540 correct=YES
$ mpirun -np 4 ./ring
ranks=4 laps=1000 messages=4000
token after last lap=3 expected=3
time=1.024 ms  per message=0.00026 ms  correct=YES
$ mpirun -np 8 ./ring
ranks=8 laps=1000 messages=8000
token after last lap=7 expected=7
time=1098.059 ms  per message=0.13726 ms  correct=YES
```

run_all.sh

```
#!/bin/bash
# builds and runs everything for Lab 08, output goes to the screen
mkdir -p ~/PDC_Lab08
cp "$(dirname "$0")"/*.c ~/PDC_Lab08/
cd ~/PDC_Lab08
. ~/mpi-env.sh
# stderr only has harmless "missing help file" warnings, so hide it
mr() { mpirun --oversubscribe "$@" 2>/dev/null; }
MR=mr

echo "### verify"
echo '$ mpicc --version'; mpicc --version | head -1
```

```
echo '$ ompi_info | grep "Open MPI:"'; ompi_info 2>/dev/null | grep "Open MPI:"
echo '$ mpirun -np 2 hostname'; mr -np 2 hostname
echo '$ nproc'; nproc

echo "### build"
for cmd in "mpicc -O2 -std=gnu11 hello_mpi.c -o hello_mpi" \
           "mpicc -O2 -std=gnu11 sum_mpi.c -o sum_mpi" \
           "gcc -O2 -fopenmp -std=gnu11 sum_omp.c -o sum_omp" \
           "mpicc -O2 -std=gnu11 master_worker.c -o master_worker" \
           "mpicc -O2 -std=gnu11 ring.c -o ring"; do
  echo "\$ $cmd"; $cmd 2>&1 && echo "  ok"
done

echo "### task1"
for n in 4 6 8; do for run in 1 2; do
  echo "\$ mpirun -np $n ./hello_mpi   (run $run)"; $MR -np $n ./hello_mpi
done; done

echo "### task2"
for n in 1 2 4 8; do echo "\$ mpirun -np $n ./sum_mpi"; $MR -np $n ./sum_mpi; done
for n in 1 2 4 8; do echo "\$ OMP_NUM_THREADS=$n ./sum_omp"; OMP_NUM_THREADS=$n ./sum_omp; done

echo "### task3"
for n in 4 8; do echo "\$ mpirun -np $n ./master_worker"; $MR -np $n ./master_worker; done
for n in 4 8; do echo "\$ mpirun -np $n ./ring"; $MR -np $n ./ring; done
```